Studio di sensibilità degli iperparametri dei metodi estrattivi

Quanto dipendono i risultati del benchmark Multi-News dalle scelte di configurazione di Centroid+MMR, LSA, clustering SBERT e LDA

Progetto finale AI for STEM, Politecnico di Torino · Report redatto l'8 ottobre 2026 · Fonti: cartella `sensibilita_estrattivi/` (driver `esegui_sensibilita.py`, notebook `analisi_sensibilita.ipynb`, `risultati/riepilogo.json`). Tutti e quattro gli step dello studio sono completati.

Questo documento riassume lo studio di sensibilità condotto sui metodi estrattivi non supervisionati del benchmark. È organizzato in tre parti. La **prima** descrive come è stato condotto il lavoro: dati, metriche, metodi, iperparametri e criteri di confronto. La **seconda** racconta i risultati principali in forma sintetica, per chi vuole solo le conclusioni. La **terza** riprende un metodo alla volta, spiegandone il funzionamento e commentandone i risultati, i punti di forza e i limiti.

Introduzione metodologica
Sintesi dei risultati
Dettaglio dei singoli metodi: Centroid+MMR, LSA, clustering SBERT, LDA

1. Introduzione metodologica

1.1 La domanda di partenza

Il benchmark confronta diciannove metodi di riassunto multi-documento sul dataset Multi-News. Undici sono *estrattivi*: costruiscono il riassunto scegliendo frasi già presenti negli articoli, senza scriverne di nuove. Ciascuno di questi metodi ha uno o più iperparametri, cioè «manopole» da fissare prima di eseguirlo: quanto penalizzare le frasi ripetitive, quanti concetti latenti cercare, in quanti gruppi dividere le frasi, quanti argomenti individuare.

Nel benchmark queste manopole sono rimaste sui loro **valori convenzionali**, cioè quelli suggeriti dalla letteratura o i default delle librerie. Nessuno di essi è stato tarato. È una scelta legittima, ma lascia aperta una domanda: *la classifica pubblicata dipende da queste scelte?* Se un metodo risultasse ultimo solo perché configurato male, il confronto sarebbe ingiusto. Lo studio risponde a questa domanda in modo **qualitativo**. Non cerca la configurazione «migliore» da pubblicare: vuole capire quali parametri contano, in che direzione agiscono e se cambiano l'ordine fra i metodi.

1.2 I dati

Lo studio usa la **split di test pulita di Multi-News**: 5.610 cluster di notizie, ottenuti dai 5.622 originali togliendo 12 righe sporche. Ogni cluster contiene da 2 a 10 articoli sullo stesso evento e un riassunto scritto da un giornalista, che fa da riferimento. Sono le stesse righe su cui sono calcolati i numeri pubblicati del benchmark. Su 22 cluster la segmentazione in frasi di pyAutoSummarizer fallisce, esattamente come nel benchmark, per cui le metriche sono calcolate su **5.588 cluster**.

1.3 I metodi analizzati

Sono stati studiati quattro metodi, per un totale di sette varianti («slug» nel benchmark), scelti fra gli undici estrattivi del benchmark. Sono quelli i cui iperparametri hanno un significato non banale e un effetto potenzialmente grande. I metodi astrattivi e gli LLM sono rimasti fuori perché un'esplorazione simile avrebbe un costo molto più alto.

Metodo	Varianti (slug)	Iperparametro studiato	Default	Valori provati
Centroid+MMR	`centroid_mmr` (TF-IDF), `centroid_mmr_bert` (MiniLM)	λ, il peso fra pertinenza e novità	0,7	0,3 · 0,5 · 0,6 · 0,7 · 0,8 · 0,9 · 1,0
LSA	`lsa` (top-k), `lsa_steinberger` (deflazione)	`K_LATENTE`, il numero di concetti latenti	11	1 · 2 · 3 · 5 · 8 · 11 · 15 · 20
Clustering SBERT	`sbert_kmeans`, `sbert_agglom`	`N_CLUSTER`, il numero di gruppi; il linkage (solo agglomerativo)	11; average	3 · 5 · 8 · 11 · 15 · 20; average · complete · single
LDA	`lda`	`N_TOPICS` × α (concentrazione dei topic); seme casuale	5 × 0,2; 42	{2, 3, 5, 8, 11} × {0,05, 0,2, 0,5, 1,0}; semi 42, 0, 1, 2, 3

In tutto sono state generate **70 configurazioni**, ciascuna sull'intera split di test. Due coincidono col default di un'altra griglia, quindi le configurazioni distinte sono 68. Ogni configurazione produce 5.588 riassunti.

Gli altri quattro estrattivi sono stati esclusi per una ragione precisa: a lunghezza bloccata non hanno iperparametri il cui studio sia informativo.

- **First-k** (`firstk_psr`, `firstk_nltk`) prende le prime frasi del documento fino a riempire lo spazio disponibile. Le due varianti differiscono solo nel modo di dividere il testo in frasi. Una volta fissata la lunghezza al riferimento, il metodo non ha nulla da regolare.
- **TextRank** e **LexRank** sono eseguiti con pyAutoSummarizer, che espone tre soli parametri. Il numero di iterazioni del PageRank serve solo alla convergenza e non cambia il risultato. Il cambio di encoder di TextRank è stato escluso per scelta, come per tutti i metodi neurali. Resta il damping `D` (0,85), che però dovrebbe avere un effetto modesto: il grafo collega ogni frase a tutte le altre, e in un grafo così l'ordine delle frasi dipende soprattutto da quanto ciascuna somiglia

alle altre, mentre `D` agisce più sull'ampiezza dei punteggi che sul loro ordine. Non è stato verificato con una corsa: è un'aspettativa basata sul funzionamento dell'algoritmo.

Le scelte che potrebbero contare davvero per TextRank e LexRank sono invece fissate dentro la libreria e non sono regolabili: l'assenza di una soglia di similarità sul grafo (prevista nel LexRank originale), la costante 0,1 aggiunta al TF-IDF di LexRank prima di calcolare le similarità, gli archi di ogni frase verso sé stessa e la mancanza di qualunque controllo della ridondanza nella scelta delle frasi. Studiarle richiederebbe di reimplementare il metodo, e i risultati descriverebbero una *variante* dell'algoritmo, non la sensibilità di quello pubblicato nel benchmark. È un possibile sviluppo futuro: la penalità anti-ridondanza, che per Centroid+MMR vale +0,012 punti, è il candidato più promettente.

1.4 Un accorgimento essenziale: la lunghezza bloccata

Molti iperparametri cambiano, come effetto collaterale, la **lunghezza** del riassunto. ROUGE tende a premiare i riassunti più lunghi: più parole si scrivono, più parole del riferimento si «indovinano». Il benchmark lo ha già mostrato: LDA sembrava competitivo solo perché produceva riassunti lunghi più del doppio degli altri. Se non si controllasse la lunghezza, lo studio misurerebbe soprattutto quanto ogni parametro allunga o accorcia il testo.

Per questo tutte le configurazioni usano il protocollo `test_budgetref` del benchmark. Ogni riassunto viene costruito fino alla **lunghezza del riassunto di riferimento** di quel cluster e poi troncato a un tetto di 1,25 volte quella lunghezza. Così, a parità di parole, una differenza fra due configurazioni misura solo la *qualità della scelta delle frasi*. In tutte le configurazioni la lunghezza media resta fra 215 e 231 parole, con una quota di riassunti entro la banda prevista compresa fra il 97,7% e il 99,4%.

1.5 Le metriche

Si usano le metriche lessicali di pyAutoSummarizer, le stesse del benchmark, che misurano quanto il riassunto prodotto si sovrappone a quello di riferimento:

- **ROUGE-1 F1**: la sovrapposizione delle singole parole. È la **metrica principale** di questo report: tutti i numeri citati senza altre indicazioni sono ROUGE-1 F1.
- **ROUGE-2 F1** e **ROUGE-L F1**: la sovrapposizione delle coppie di parole consecutive e della più lunga sottosequenza comune. Servono a confermare le indicazioni di ROUGE-1.
- **BLEU** e **METEOR**: riportate per completezza.

Lo studio non usa metriche basate su LLM (G-Eval) né BERTScore, perché un'esplorazione di 70 configurazioni deve restare economica. L'implementazione di ROUGE di pyAutoSummarizer conta gli n-grammi unici, quindi i valori non sono confrontabili con quelli della letteratura, ma lo sono fra loro.

Per orientarsi sulla scala: nella classifica pubblicata a lunghezza bloccata, fra il miglior estrattivo (`centroid_mmr`, 0,378) e il peggiore (`lda`, 0,338) ci sono 0,040 punti di ROUGE-1. Uno spostamento di 0,001-0,002 è quindi trascurabile, uno di 0,010 è rilevante.

1.6 Come si confrontano le configurazioni

Ogni configurazione viene confrontata con il **default del benchmark** dello stesso metodo, con cinque strumenti:

- **Differenza appaiata.** Per ciascuno dei 5.588 cluster si calcola la differenza di metrica fra la configurazione e il default, e poi la si media. Poiché i cluster sono gli stessi, la grande variabilità fra cluster facili e difficili si annulla e anche differenze piccole diventano misurabili.
- **Intervallo di confidenza al 95%**, ottenuto con 2.000 ricampionamenti bootstrap. Una differenza si considera *significativa* quando l'intervallo non contiene lo zero.
- **Similarità di Jaccard** fra le frasi scelte e quelle scelte dal default. Dice *quanto cambia la selezione*: 1 significa le stesse frasi, 0 nessuna in comune.
- **Ridondanza interna**, cioè la similarità media fra le frasi scelte. Dice se il riassunto ripete le stesse informazioni.
- **Parole per frase scelta.** Dice se il metodo predilige frasi lunghe e ricche o frasi brevi e periferiche.

Gli ultimi tre strumenti servono a spiegare *perché* le metriche si muovono, oltre a misurare *quanto*.

1.7 Affidabilità della pipeline

Lo studio usa un driver separato (`esegui_sensibilita.py`) che contiene una **copia letterale** delle funzioni di generazione dei notebook del benchmark. Come controllo, la configurazione di default deve riprodurre i risultati già pubblicati, ed è così:

Variante (default)	Riassunti identici ai pubblicati	Scarto massimo sulle medie
`centroid_mmr`, `lsa`, `lsa_steinberger`, `lda`	5.588 / 5.588	0
`centroid_mmr_bert`	5.555 / 5.588	0,0002
`sbert_kmeans`	4.135 / 5.588	0,0007
`sbert_agglom`	3.032 / 5.588	0,0003

I metodi che usano TF-IDF o conteggi di parole riproducono il pubblicato in modo esatto. Quelli che usano gli embedding neurali MiniLM no, perché gli embedding ricalcolati su CPU differiscono dagli originali nelle ultime cifre decimali. Questo basta a scambiare frasi quasi identiche, ad esempio lo stesso lancio d'agenzia ripreso da due testate. Le metriche medie però coincidono entro 0,0007, quindi i confronti restano validi. Il fenomeno è approfondito nella sezione sul clustering SBERT.

> **Nota: perché tanti riassunti diversi danno uno scarto così piccolo**
>
> Può sembrare strano che `sbert_agglom` riproduca solo il 54% dei riassunti pubblicati e abbia comunque uno scarto di appena 0,0003. Sembra strano anche che `sbert_kmeans`, con più riassunti identici (74%), abbia lo scarto più alto (0,0007). Il confronto riga per riga fra le due versioni chiarisce entrambi i punti.
>
	`sbert_kmeans`	`sbert_agglom`
righe con testo diverso dal pubblicato	1.453 (26%)	2.556 (46%)
parole in comune fra le due versioni (mediana, righe diverse)	85%	84%
variazione tipica di ROUGE-1 su una riga diversa	±0,010	±0,010
righe che peggiorano / migliorano	45% / 45%	47% / 49%
scarto delle medie su ROUGE-1 (errore standard)	+0,00014 (0,00021)	+0,00021 (0,00024)
>
> **Le differenze si compensano.** Nelle righe diverse cambia in genere una sola frase, sostituita da una quasi identica. Il singolo riassunto guadagna o perde circa 0,01 punti, ma con la stessa probabilità in un senso o nell'altro, quindi sulla media dei 5.588 cluster le variazioni si annullano. Più righe diverse non producono uno scarto più grande, solo più variazioni casuali che si compensano.
>
> **Gli scarti residui sono rumore.** Con variazioni di questa ampiezza la media oscilla di circa ±0,0002-0,0003, ed è quello che si osserva. Lo 0,0007 di `sbert_kmeans` è il massimo fra cinque metriche e viene da BLEU, la più rumorosa. Su ROUGE-1 il suo scarto (0,00014) è anzi più piccolo di quello dell'agglomerativo. La differenza fra i due scarti non dice quindi nulla sui due metodi.
>
> **Perché l'agglomerativo cambia più righe** (interpretazione). Il clustering agglomerativo è una catena di fusioni, ciascuna dipendente dalle precedenti, quindi un'inversione minima fra due frasi quasi identiche si propaga all'intero albero. Il k-means invece sceglie la migliore di 10 inizializzazioni e converge verso partizioni più stabili. In entrambi pesano i pareggi del medoide: nei gruppi di due frasi il pareggio è esatto e lo decide l'arrotondamento numerico.

> **Come leggere questi risultati**
>
> L'esplorazione è stata condotta **sul test**. Un valore che qui risulta «migliore» è stato scelto guardando proprio i dati su cui viene misurato, quindi non va presentato come la prestazione reale del metodo: per una taratura vera andrebbe scelto su un set di validazione. Inoltre, a lunghezza bloccata i numeri misurano la capacità di *selezionare i contenuti* a parità di spazio, non la prestazione in un uso reale. **I numeri pubblicati del benchmark non cambiano**: lo studio non ha scritto nulla nella cartella `results/`.

2. Sintesi dei risultati

In una frase: **la classifica del benchmark regge**. Gli iperparametri spostano le metriche, a volte in modo apprezzabile, ma i default convenzionali sono quasi sempre già vicini al meglio. Nessun confronto fra varianti dello stesso metodo si ribalta, e i metodi in testa e in coda restano dove sono. Il grafico riassume lo studio: per ogni variante, la barra grigia copre tutti i valori di ROUGE-1 ottenuti nella griglia, il cerchio vuoto è il valore pubblicato e il punto verde il migliore trovato.

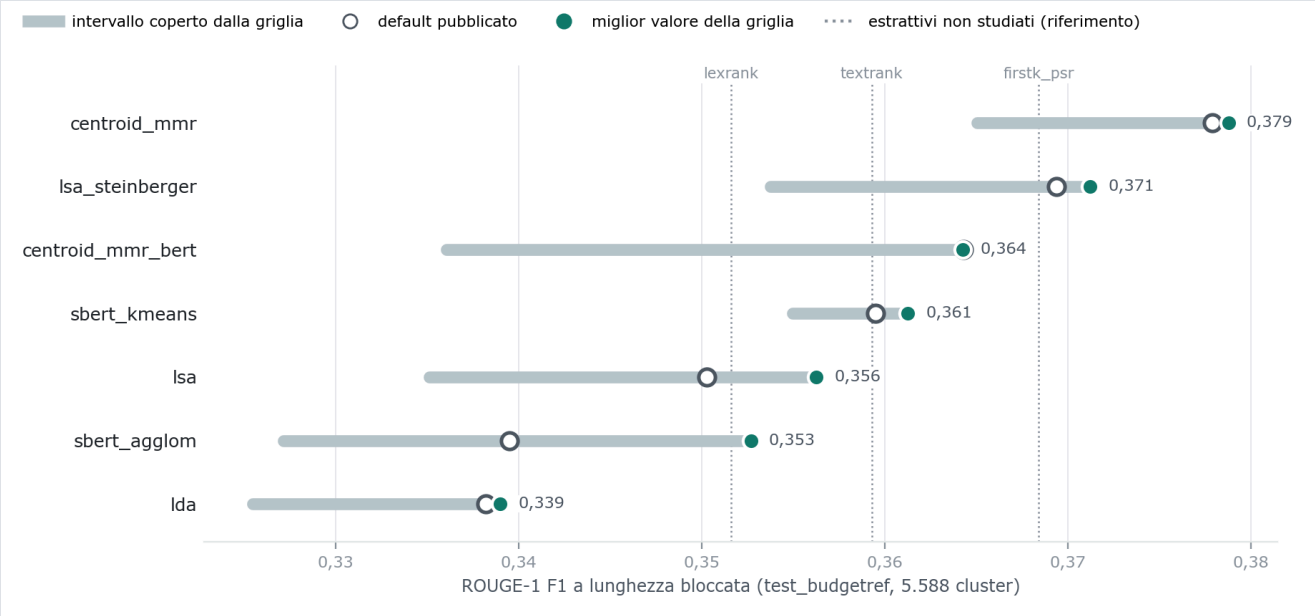


Figura 1 — Sensibilità di ciascuna variante ai propri iperparametri. Barre lunghe = metodo sensibile; cerchio vuoto lontano dal punto verde = default migliorabile. Le linee punteggiate indicano tre estrattivi non studiati, come riferimento.

Dal grafico e dalle tabelle emergono sei messaggi.

> **1. Nella maggior parte dei casi il default è già (quasi) il migliore.** Per `centroid_mmr`, `centroid_mmr_bert`, `lsa_steinberger`, `sbert_kmeans` e `lda`, il miglior valore della griglia supera il default di 0,002 al massimo, cioè di una quantità trascurabile. Spesso la differenza non è nemmeno statisticamente distinguibile da zero. Le scelte «di convenzione» si sono rivelate ragionevoli.

> **2. Ci sono due eccezioni, ed entrambe hanno una spiegazione chiara.** La prima è `sbert_agglom`: il collegamento `average` scelto per il clustering agglomerativo è sfavorevole, e passando a `complete` si guadagnano +0,013 punti, il miglioramento più grande dello studio. La seconda è `lsa` nella variante top-k: 11 dimensioni latenti sono troppe, con una sola si guadagnano +0,006 punti.

> **3. Quello che conta di più è la regola di selezione, non il numero.** Gli effetti più grandi non vengono dal «girare una manopola numerica» ma dal cambiare il modo in cui il metodo sceglie le frasi: il tipo di collegamento nel clustering, l'eliminazione o meno della ridondanza in LSA. Le variazioni numeriche moderate attorno al default spostano ROUGE-1 di pochi millesimi; solo i valori estremi della griglia causano perdite di 0,01-0,03.

> **4. Cambiano le frasi scelte, non la qualità.** Fra valori vicini di un parametro cambia da un terzo a metà delle frasi selezionate, eppure ROUGE-1 si muove di pochi millesimi. In un cluster di notizie molte frasi sono quasi intercambiabili, perché più articoli raccontano gli stessi fatti. Per lo stesso motivo LDA sceglie frasi diverse nel 98% dei cluster se si cambia solo il seme casuale, senza che le metriche ne risentano.

> **5. I confronti interni a ogni metodo restano tutti validi.** TF-IDF batte gli embedding BERT in Centroid+MMR per ogni valore di λ. La variante di LSA con deflazione batte la top-k per ogni numero di dimensioni. Il k-means batte l'agglomerativo con qualunque collegamento, anche se con `complete` il divario si dimezza (da 0,020 a 0,009).

> **6. La classifica degli estrattivi cambia solo nella parte bassa.** Anche dando a ogni metodo la sua configurazione migliore, i primi sei posti restano gli stessi, con `centroid_mmr` sempre primo. In coda `lsa` e `sbert_agglom` superano `lexrank`. **`lda` resta ultimo in ogni configurazione provata**: la sua debolezza è strutturale e non dipende dalla taratura.

Variante	Default pubblicato	Miglior valore (configurazione)	Guadagno	Caso peggiore (configurazione)	Il default era buono?
`centroid_mmr`	0,378	0,379 (λ = 0,6)	+0,001 n.s.	0,365 (λ = 0,3)	sì
`lsa_steinberger`	0,369	0,371 (K = 8)	+0,002	0,354 (K = 20)	sì
`centroid_mmr_bert`	0,364	0,364 (λ = 0,7)	0	0,336 (λ = 0,3)	sì, è l'ottimo
`sbert_kmeans`	0,360	0,361 (N = 15 o 20)	+0,002	0,355 (N = 3)	sì
`lsa`	0,350	0,356 (K = 1)	+0,006	0,335 (K = 20)	**no**
`sbert_agglom`	0,340	0,353 (complete)	+0,013	0,327 (single)	**no**, per il linkage
`lda`	0,338	0,339 (8 topic, α = 0,05)	+0,001 n.s.	0,325 (2 topic, α = 1)	sì

ROUGE-1 F1 a lunghezza bloccata, 5.588 cluster. «n.s.» = l'intervallo di confidenza della differenza contiene lo zero. Riferimenti non studiati: `firstk_*` 0,368, `textrank` 0,359, `lexrank` 0,352.

3. Dettaglio dei singoli metodi

3.1 Centroid+MMR (notebook 11)

Come funziona

Il metodo rappresenta ogni frase come un vettore numerico, con TF-IDF (pesi delle parole) nella variante `centroid_mmr` o con un embedding neurale MiniLM in `centroid_mmr_bert`. Calcola poi il **centroide** del cluster, cioè il «vettore medio» che rappresenta l'argomento centrale delle notizie. Costruisce il riassunto una frase alla volta, scegliendo ogni volta quella che massimizza un compromesso fra due esigenze:

- la **pertinenza**, cioè quanto la frase è vicina al centroide;
- la **novità**, cioè quanto la frase è diversa da quelle già scelte (è la tecnica MMR, *Maximal Marginal Relevance*).

Il parametro **λ** regola il peso fra le due: con λ = 1 conta solo la pertinenza (è il metodo «Centroid» classico, senza controllo della ridondanza), con λ basso conta soprattutto la novità. Il benchmark usa λ = 0,7.

Cosa è emerso

Il default λ = 0,7 si trova già nella zona migliore, per entrambe le varianti. Con TF-IDF le prestazioni formano un **altopiano fra 0,5 e 0,7**, dove le differenze non sono distinguibili da zero; con BERT 0,7 è esattamente il massimo.

Il controllo della ridondanza serve. Togliere l'MMR (λ = 1) costa 0,012 punti con TF-IDF e 0,005 con BERT: senza penalità il centroide attira frasi che ripetono la stessa informazione, e infatti la ridondanza interna sale da 0,10 a 0,17. Esagerare con la novità è però peggio che non averne. Con λ = 0,3 il metodo, pur di evitare ripetizioni, sceglie frasi brevi e periferiche (19-22 parole contro 32-33), povere di contenuto centrale, e perde 0,013 punti con TF-IDF e ben 0,028 con BERT.

Le due varianti reagiscono a λ in modo diverso, e la ragione è istruttiva. Con gli embedding neurali tutte le frasi dello stesso cluster sono già molto simili fra loro (similarità media circa 0,5, contro circa 0,1 con TF-IDF). Il termine di novità è quindi sempre grande e, appena gli si dà peso, prevale sulla pertinenza. Ne segue che **lo stesso valore di λ non ha lo stesso significato** nei due spazi di rappresentazione: non esiste un λ «universale».

λ	TF-IDF: ROUGE-1	differenza dal default [IC 95%]	BERT: ROUGE-1	differenza dal default [IC 95%]
0,3	0,3651	−0,0129 [−0,0145; −0,0112]	0,3361	−0,0282 [−0,0299; −0,0263]
0,5	0,3782	+0,0003 [−0,0011; +0,0016]	0,3575	−0,0067 [−0,0082; −0,0052]
0,6	0,3788	+0,0009 [−0,0000; +0,0018]	0,3635	−0,0007 [−0,0018; +0,0003]
0,7	**0,3779**	**default**	**0,3643**	**default**
0,8	0,3751	−0,0028 [−0,0037; −0,0020]	0,3628	−0,0014 [−0,0023; −0,0005]
0,9	0,3708	−0,0071 [−0,0081; −0,0060]	0,3617	−0,0025 [−0,0036; −0,0014]
1,0	0,3655	−0,0124 [−0,0138; −0,0111]	0,3594	−0,0048 [−0,0060; −0,0036]

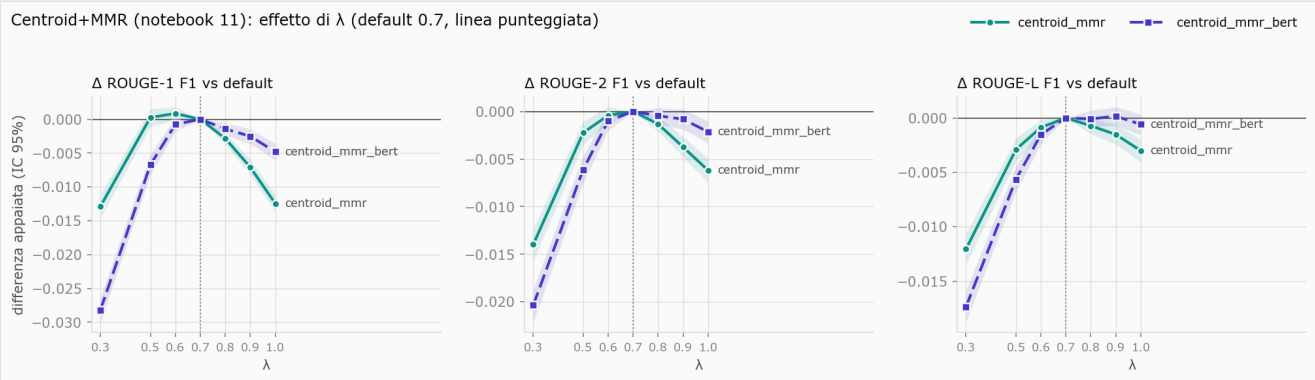


Figura 2 — Differenza rispetto a λ = 0,7 per le tre F1 ROUGE (banda colorata: intervallo di confidenza al 95%). La curva a campana conferma che il default sta sul massimo.

Punti di forza

- È il miglior estrattivo del benchmark per ogni λ fra 0,5 e 0,9.
- Un solo parametro, interpretabile, con un'ampia zona di valori buoni.
- La variante TF-IDF è deterministica e si riproduce esattamente.

Limiti

- Ai valori estremi di λ perde oltre 0,01 punti.
- Il significato di λ dipende dalla rappresentazione: va ritarato se si cambia encoder.
- La variante BERT è più fragile verso il basso ed è sempre inferiore a TF-IDF.

Per il confronto complessivo

Il miglior `centroid_mmr_bert` (0,364) resta sotto `centroid_mmr` per **ogni** λ provato: il vantaggio di TF-IDF su BERT non dipende dalla scelta di λ. `centroid_mmr` resta il primo degli estrattivi in tutta la griglia, salvo ai due estremi (λ = 0,3 e λ = 1), dove scende a 0,365.

3.2 LSA, Latent Semantic Analysis (notebook 15)

Come funziona

LSA costruisce una tabella che incrocia le frasi con le parole del cluster (pesi TF-IDF) e la scompone con la **SVD** (decomposizione ai valori singolari). La scomposizione individua un certo numero di **«concetti latenti»**: direzioni lungo cui molte frasi e parole variano insieme e che corrispondono, grosso modo, ai temi del cluster. Il parametro `K_LATENTE` stabilisce quanti di questi concetti tenere; il benchmark ne usa 11 per convenzione. Le due varianti condividono la scomposizione ma scelgono le frasi in modo diverso:

- `lsa` (*top-k*) prende semplicemente le frasi più «forti» nello spazio dei concetti, cioè quelle con il vettore latente più lungo;
- `lsa_steinberger` (*deflazione*) sceglie una frase alla volta e, dopo ogni scelta, rimuove dallo spazio il concetto che quella frase ha già coperto, così che la frase successiva debba portare qualcosa di nuovo. È un meccanismo anti-ridondanza.

Cosa è emerso

Le due varianti reagiscono a `K_LATENTE` in modo opposto, ed è il risultato più interessante. `lsa` **preferisce pochi concetti**: con K da 1 a 5 guadagna circa 0,005 punti, poi peggiora. Con un solo concetto la «forza» di una frase coincide con la sua vicinanza al tema dominante, quindi vengono scelte frasi più lunghe e centrali (28 parole contro 21). Queste frasi sono anche più ripetitive (ridondanza 0,19 contro 0,10), ma a parità di spazio il contenuto centrale vale più della varietà.

`lsa_steinberger` **invece ha bisogno di molti concetti**, perché per questa variante K è anche il numero di passi di deflazione, cioè quanto a lungo il metodo continua a cercare informazione nuova. Con K = 1 le due varianti producono esattamente lo stesso riassunto. Al crescere di K la ridondanza scende in modo regolare (da 0,19 a 0,03) e la qualità sale fino a un massimo a K = 8, appena sopra il default.

Oltre 11 concetti peggiorano entrambe le varianti, allo stesso modo (−0,006 a K = 15, −0,015 a K = 20). I concetti in più sono deboli: spesso corrispondono a una singola frase isolata che non condivide parole con le altre, e premiandoli il metodo finisce per scegliere frasi corte e marginali.

Infine le metriche non sono del tutto concordi sul massimo di `lsa_steinberger`: ROUGE lo colloca a K = 8, METEOR e BLEU a K = 11. Il default è quindi sostanzialmente ottimale per questa variante.

K	`lsa` : ROUGE-1	differenza [IC 95%]	ridondanza	`steinberger` : ROUGE-1	differenza [IC 95%]	ridondanza
1	0,3563	+0,0059 [+0,0040; +0,0081]	0,194	0,3563	−0,0131 [−0,0152; −0,0110]	0,194
3	0,3549	+0,0046 [+0,0029; +0,0064]	0,153	0,3602	−0,0092 [−0,0111; −0,0073]	0,130
5	0,3549	+0,0045 [+0,0030; +0,0061]	0,131	0,3651	−0,0043 [−0,0060; −0,0026]	0,088
8	0,3534	+0,0030 [+0,0018; +0,0042]	0,113	0,3712	+0,0018 [+0,0006; +0,0031]	0,051
11	**0,3503**	**default**	**0,100**	**0,3694**	**default**	**0,032**
15	0,3442	−0,0062 [−0,0074; −0,0049]	0,089	0,3635	−0,0059 [−0,0071; −0,0046]	0,022
20	0,3351	−0,0152 [−0,0168; −0,0136]	0,076	0,3538	−0,0156 [−0,0173; −0,0140]	0,017

Per brevità è omesso K = 2, che si comporta come K = 1 e K = 3.

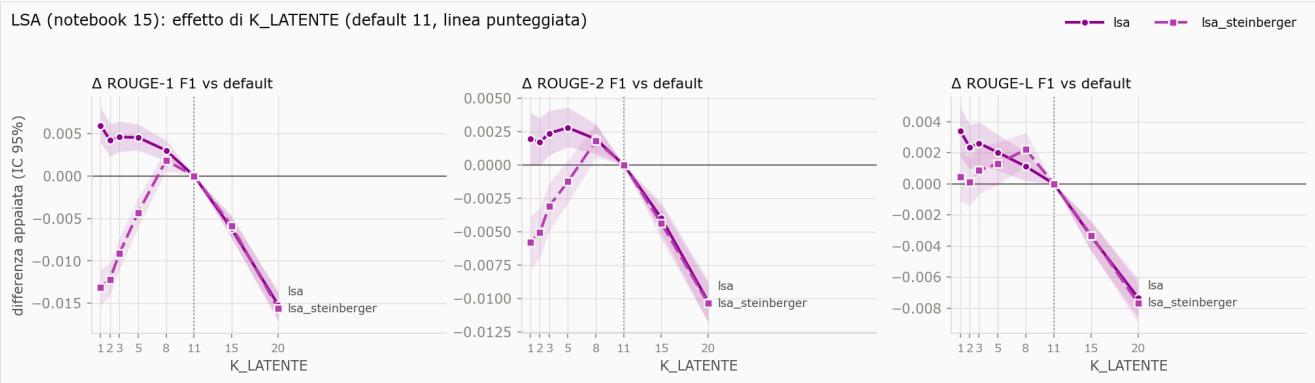


Figura 3 — Differenza rispetto a K = 11. Le due curve si incrociano: sotto il default `lsa` migliora e `lsa_steinberger` peggiora; sopra peggiorano entrambe allo stesso modo.

Punti di forza

- Metodo deterministico, veloce, senza reti neurali.
- La deflazione è robusta: attorno al default (K tra 8 e 11) varia di appena 0,002.
- Il comportamento è spiegabile in modo trasparente con la ridondanza.

Limiti

- La variante top-k è sensibile a K, e il default convenzionale è sfavorevole.
- Troppe dimensioni introducono «concetti» di rumore.
- Le diverse metriche non concordano esattamente sull'ottimo.

Per il confronto complessivo

Il default K = 11 favoriva `lsa_steinberger` rispetto a `lsa`. Scegliendo per ciascuna il K migliore, il divario fra le due scende da 0,019 a 0,015, ma **la deflazione resta nettamente superiore**. `lsa_steinberger` è già secondo fra gli estrattivi con il default (0,369, appena sopra `firstk_*` a 0,368) e con K = 8 consolida il posto (0,371). `lsa` con K = 1 (0,356) supererebbe `lexrank` (0,352). Sono spostamenti di pochi millesimi.

3.3 Clustering SBERT (notebook 16)

Come funziona

Il metodo trasforma ogni frase in un embedding neurale MiniLM e raggruppa le frasi simili in `N_CLUSTER` gruppi, che rappresentano i sottotemi del cluster di notizie. Per formare i gruppi usa il **k-means** (`sbert_kmeans`) oppure un clustering **agglomerativo** (`sbert_agglom`), che parte dalle singole frasi e le fonde a coppie. Da ogni gruppo prende la frase più rappresentativa, il **medoide**. Le frasi entrano nel riassunto «a giri»: prima i medoidi di tutti i gruppi, dal più numeroso al più piccolo, poi i secondi rappresentanti, e così via fino a esaurire lo spazio disponibile.

Nell'agglomerativo conta anche il **linkage**, cioè il criterio per decidere quanto due gruppi sono vicini: `single` guarda la coppia di frasi più vicina, `complete` la più lontana, `average` (il default) la media.

Cosa è emerso

Il linkage conta molto più del numero di gruppi. Passare da `average` a `complete` fa guadagnare a `sbert_agglom` 0,013 punti, il doppio di qualunque variazione di `N_CLUSTER`; `single` ne perde altrettanti. L'ordine `complete` > `average` > `single` ha una spiegazione intuitiva. Il legame singolo tende a «concatenare» quasi tutte le frasi in un unico gruppo gigante e lascia isolate le frasi anomale. Il legame completo forma gruppi compatti ed equilibrati. Il medio sta in mezzo.

I gruppi sbilanciati fanno male per via della selezione a giri. Ogni gruppo, anche se contiene una sola frase anomala (un titolo, una didascalia, una frase fuori tema), porta il suo medoide nel riassunto al primo giro, prima dei secondi rappresentanti dei temi veri. Lo si vede dalla lunghezza delle frasi scelte: 19-20 parole con `average` e `single`, 23 con `complete`, 25 con il k-means, che per costruzione forma gruppi di dimensione comparabile.

Il numero di gruppi conta poco. Il k-means è quasi indifferente a `N_CLUSTER`: guadagna 0,002 con 15-20 gruppi e perde 0,005 con 3-5. L'agglomerativo ha un andamento a U, con il default 11 vicino al punto peggiore. Con pochi gruppi le frasi isolate sono poche; con molti (20) lo spazio si esaurisce già durante il primo giro, prima di arrivare ai gruppi più piccoli, che sono proprio quelli delle frasi isolate.

N_CLUSTER	kmeans : ROUGE-1	differenza [IC 95%]	agglom : ROUGE-1	differenza [IC 95%]
3	0,3550	−0,0047 [−0,0063; −0,0030]	0,3440	+0,0043 [+0,0026; +0,0061]
5	0,3551	−0,0046 [−0,0061; −0,0030]	0,3387	−0,0010 [−0,0026; +0,0006]
8	0,3583	−0,0013 [−0,0027; +0,0001]	0,3378	−0,0019 [−0,0031; −0,0008]
11	**0,3596**	**default**	**0,3397**	**default**
15	0,3613	+0,0016 [+0,0002; +0,0031]	0,3427	+0,0031 [+0,0017; +0,0043]
20	0,3613	+0,0016 [+0,0001; +0,0032]	0,3461	+0,0064 [+0,0047; +0,0079]

Linkage (agglomerativo, N = 11)	ROUGE-1	differenza [IC 95%]	ROUGE-2: differenza	parole per frase
average (default)	**0,3397**	**default**	**default**	**19,9**
complete	0,3527	+0,0130 [+0,0115; +0,0144]	+0,0075	23,2
single	0,3272	−0,0125 [−0,0140; −0,0110]	−0,0079	19,1

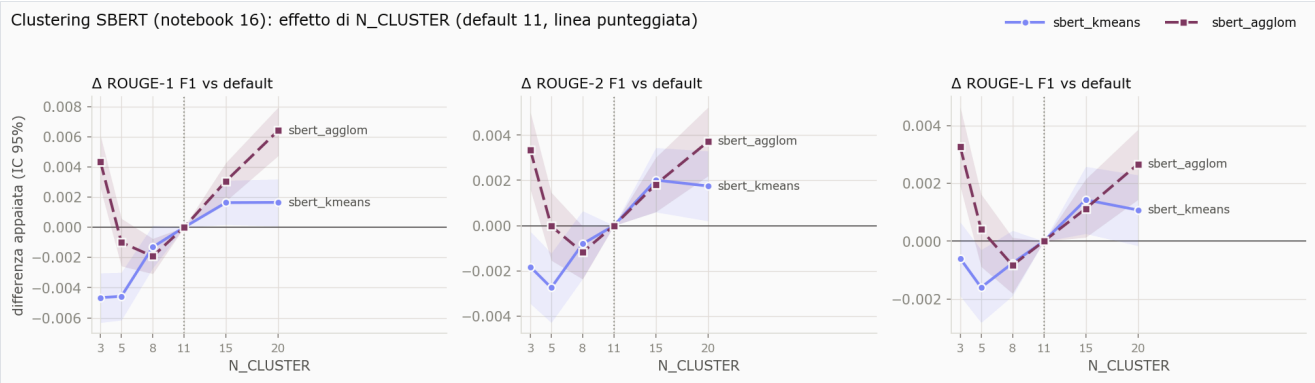


Figura 4 — Differenza rispetto a N_CLUSTER = 11. Il k-means resta entro pochi millesimi dal default; l'agglomerativo mostra l'andamento a U.

Un metodo numericamente instabile, ma solo in superficie

Basta un rumore di un milionesimo sugli embedding perché il metodo scelga frasi diverse in circa metà dei cluster. Per questo i riassunti ricalcolati in questo studio coincidono con quelli pubblicati solo nel 74% dei casi per il k-means e nel 54% per l'agglomerativo. Gli scambi avvengono però quasi sempre fra frasi praticamente identiche, come lo stesso lancio d'agenzia ripreso da più testate, e le metriche medie non si muovono (scarto massimo 0,0007). L'instabilità riguarda il *testo* prodotto, non la sua *qualità*. Il meccanismo è spiegato in dettaglio nella nota della sezione 1.7.

Punti di forza

- Il k-means è robusto: il numero di gruppi incide per pochi millesimi.
- Gli embedding neurali catturano somiglianze di significato, non solo di parole.
- L'idea dei sottotemi ha un'interpretazione naturale.

Limiti

- Per l'agglomerativo il linkage è decisivo, e il default `average` è sfavorevole.
- Le frasi anomale isolate entrano nel riassunto a scapito dei temi veri.
- La selezione esatta non è riproducibile su hardware diverso.

Per il confronto complessivo

Il divario pubblicato fra k-means e agglomerativo (0,020) è **per più di metà un effetto del linkage** `average`: con `complete` scende a 0,009, ma il k-means resta davanti. Con `complete`, `sbert_agglom` (0,353) risalirebbe dal penultimo posto fra gli estrattivi, superando `lsa` e `lexrank`. È lo spostamento di classifica più grande dell'intero studio.

3.4 LDA, Latent Dirichlet Allocation (notebook 17)

Come funziona

LDA è un modello probabilistico di **topic**: ipotizza che ogni testo sia una miscela di pochi argomenti e che ogni argomento sia caratterizzato da certe parole. Nel benchmark ogni frase del cluster è trattata come un piccolo documento. Il modello, addestrato sui conteggi delle parole, stima per ogni frase la sua miscela di argomenti. Lo spazio del riassunto viene poi diviso fra gli argomenti **in proporzione al loro peso** complessivo, e per ogni argomento si prendono le frasi che gli appartengono di più, fino a riempire la sua quota. Gli iperparametri studiati sono tre:

- `N_TOPICS`, il numero di argomenti (default 5);
- **α**, che controlla quanto ogni frase è «specializzata». Con α piccolo ogni frase appartiene quasi tutta a un solo argomento, con α grande le miscele sono sfumate. Nel benchmark vale 1/`N_TOPICS` = 0,2. Nella griglia è stato separato dal numero di argomenti, tenendo fisso a 0,2 l'analogo parametro η per le parole;
- il **seme casuale**, perché l'addestramento parte da un'inizializzazione casuale. Il benchmark usa 42; qui sono stati provati altri quattro semi per misurare quanto pesa il caso.

Cosa è emerso

Il default è già nella zona migliore. Nessuna delle 19 alternative lo supera in modo significativo. Le due che fanno leggermente meglio (α = 0,05 con 5 oppure 8 argomenti) guadagnano meno di un millesimo, con intervalli di confidenza che comprendono lo zero.

Il parametro che conta è α, e conviene tenerlo basso. Per ogni numero di argomenti, aumentare α peggiora i risultati in modo regolare: con 5 argomenti si perdono 0,004 punti ad α = 0,5 e 0,007 ad α = 1, fino a −0,013 con 2 argomenti e α = 1. Quando le miscele sono sfumate ogni frase «appartiene un po' a tutti gli argomenti», la divisione dello spazio fra argomenti perde significato e la selezione diventa meno mirata. Questo vale anche se, con α alto, la ridondanza interna scende (da 0,095 a 0,067): anche qui la varietà non compensa la perdita di contenuto centrale.

Il numero di argomenti conta meno. Con 2-3 argomenti si perdono 0,003-0,005 punti, fra 5 e 8 c'è un altopiano, con 11 si perde 0,002. Più argomenti significano frasi scelte più corte e più numerose: da 50 parole per frase con 2 argomenti a 31-34 con 11.

Il seme cambia le frasi, non la qualità. È il risultato più sorprendente. Cambiando solo il seme casuale, con tutti gli altri parametri fermi, LDA sceglie un insieme di frasi diverso nel **98% dei cluster**: la Jaccard con il default è appena 0,32. Eppure ROUGE-1 si sposta al massimo di 0,0004, con intervalli che comprendono lo zero. Un cluster ha poche decine di frasi, troppo poche perché un modello di topic trovi una soluzione stabile, e ogni inizializzazione porta a una soluzione diversa ma ugualmente plausibile. Ne segue che le Jaccard molto basse viste nella griglia (0,22-0,42) misurano soprattutto questa instabilità e non l'effetto del parametro.

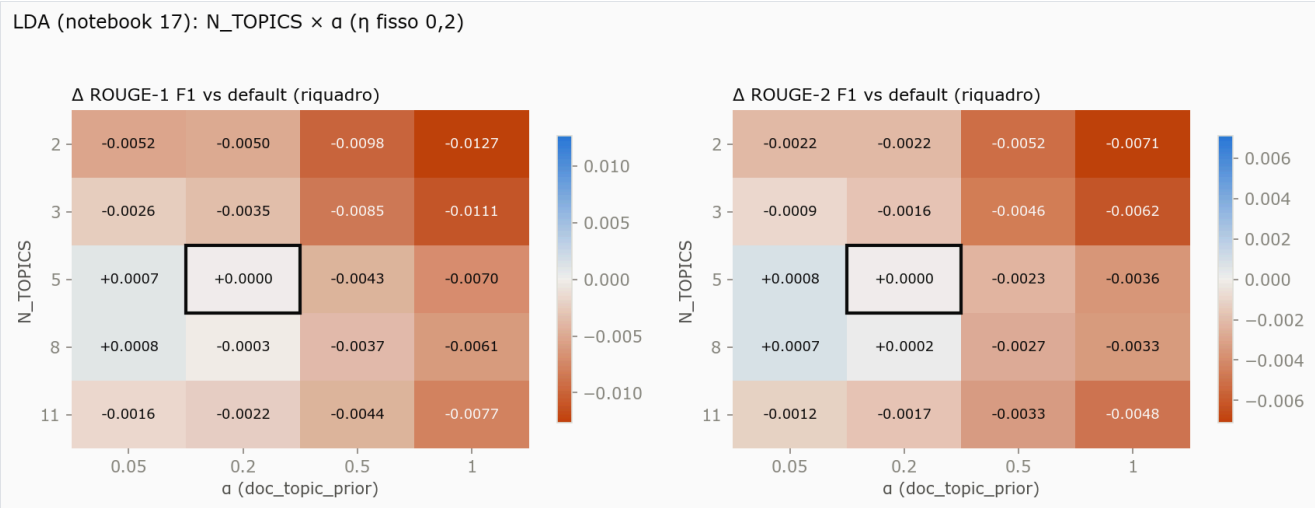


Figura 5 — Differenza rispetto al default (riquadro: 5 argomenti, α = 0,2) per ogni combinazione di N_TOPICS e α. Il colore si scurisce verso destra (α alto) e verso l'alto (pochi argomenti): sono le due direzioni in cui il metodo peggiora.

Seme casuale	ROUGE-1	differenza [IC 95%]	Jaccard con il default	cluster con frasi identiche
42 (default)	**0,3382**	**default**	**1**	**100%**
0	0,3384	+0,0002 [−0,0014; +0,0018]	0,32	2%
1	0,3385	+0,0003 [−0,0013; +0,0020]	0,32	2%
2	0,3377	−0,0004 [−0,0022; +0,0012]	0,32	2%
3	0,3379	−0,0003 [−0,0019; +0,0014]	0,32	2%

Va segnalata anche un'altra caratteristica del metodo. **LDA sceglie frasi molto lunghe**, circa 42 parole in media contro le 20-33 degli altri estrattivi, e questo resta vero in tutta la griglia. Verosimilmente una frase lunga contiene più parole «tematiche», ottiene un'appartenenza più netta a un argomento e finisce in cima alla sua graduatoria. A lunghezza bloccata il riassunto contiene così poche frasi (4-7) e copre meno aspetti della notizia. Per lo stesso motivo la frase a cavallo del limite pesa di più, e i riassunti di LDA risultano leggermente più lunghi degli altri (220-231 parole contro 215).

Il METEOR medio di LDA non è utilizzabile: in circa metà delle configurazioni, default compreso, è falsato da un unico cluster degenere (riga 50540, un elenco di birrifici) per cui la formula di pyAutoSummarizer produce un valore enorme e negativo. È un problema già documentato nel benchmark; ROUGE e BLEU non ne risentono.

Punti di forza

- Il default convenzionale è già ottimale.
- Le metriche sono del tutto insensibili al seme casuale.
- Riproduce esattamente i risultati pubblicati.
- Gli argomenti sono interpretabili tramite le loro parole caratteristiche.

Limiti

- Resta l'ultimo degli estrattivi con qualunque configurazione.
- La selezione delle frasi è instabile e cambia completamente con il seme.
- Predilige frasi lunghe, quindi riassunti con poche frasi.
- Un α alto peggiora sistematicamente i risultati.

Per il confronto complessivo

Il benchmark aveva già mostrato che il buon richiamo di LDA nell'ambito `test` dipendeva dalla lunghezza dei suoi riassunti, e che a lunghezza bloccata il metodo scende all'ultimo posto fra gli estrattivi (0,338). Questo studio aggiunge che **l'ultimo posto non dipende dalla taratura**: il miglior LDA della griglia (0,339) resta sotto tutti gli altri estrattivi, anche nelle loro configurazioni di default. La debolezza è strutturale. Un modello di topic stimato sulle poche decine di frasi di un singolo cluster è instabile, e la sua preferenza per le frasi lunghe si adatta male a uno spazio limitato.

3.5 Considerazioni conclusive

Guardati insieme, i quattro metodi raccontano una storia coerente. A lunghezza fissata, la qualità di un riassunto estrattivo dipende soprattutto dall'**equilibrio fra contenuto centrale e varietà**. Un po' di controllo della ridondanza aiuta: lo mostrano l'MMR a $\lambda = 0{,}7$ e la deflazione di LSA, che rende `lsa_steinberger` il secondo estrattivo del benchmark. Ma quando un parametro spinge il metodo a premiare le frasi *perché isolate* dalle altre, il danno supera il beneficio. Succede con λ troppo basso in Centroid+MMR, con troppe dimensioni in LSA e con il linkage `single` o `average` nel clustering. Il segnale è sempre lo stesso: la lunghezza media delle frasi scelte cala, perché le frasi isolate sono tipicamente brevi e periferiche. Conta il motivo per cui una frase viene scelta, non la sua lunghezza in sé. Il caso opposto è LDA, la cui preferenza per frasi lunghissime lo porta a coprire pochi aspetti della notizia.

Gli iperparametri agiscono proprio spostando questi equilibri, e i default convenzionali li tengono quasi sempre in una posizione ragionevole. Le uniche correzioni che valga la pena considerare sono due: il linkage `complete` per `sbert_agglom` e un K piccolo per `lsa`. In un eventuale lavoro futuro andrebbero scelte su un set di validazione, non sul test, prima di poterle adottare nel benchmark. Per tutto il resto, **le conclusioni del benchmark pubblicato non dipendono dalla scelta degli iperparametri**.

Fuori dal perimetro dello studio, per scelta: il damping di TextRank e LexRank, il parametro η di LDA, First-k e il cambio di encoder per i metodi neurali. Lo studio inoltre misura solo metriche lessicali di sovrapposizione: non dice come cambierebbe il giudizio di un valutatore LLM (G-Eval), che nel benchmark si è dimostrato poco sensibile alla lunghezza e diversamente orientato rispetto a ROUGE.